

知ってた？

JavaScriptの " 正しさ " を検証する  
テストが 5万以上もあること

(Test262)

西 悠太 / 株式会社ダイニー

# 自己紹介

西 悠太 (Nishi Yuta)

株式会社ダイニー / Platform Engineer

TypeScriptが大好きです。  
V8も大好きです。

@riya-amemiya



今日のゴール

JavaScript が  
「どのブラウザでも同じように動く」  
その"当たり前"を、  
誰が守っているのか？

...を知って、へー！と思って帰ってもらおう。

# 01

JavaScriptの仕様は  
誰が決めているのか

# ECMAScript と TC39

私たちが書いている JavaScript の言語仕様は **ECMAScript**。  
これを制定しているのは **Ecma International** という標準化団体。

その中の **TC39** (Technical Committee 39) が ECMAScript の仕様策定を担当。

Google ・ Apple ・ Mozilla ・ Microsoft といったブラウザベンダーに加え、  
Bloomberg ・ Igalia などの企業、招待された個人の専門家も参加。  
特定の企業ではなく、複数の関係者による合議で決まる。

早速、質問です 🙋

```
console.log("Hello, World!");
```

Q. これ、皆さんのブラウザで実行したらどうなりますか？

```
console.log("Hello, World!");
```

**Chrome** Hello, World!

**Safari** Hello, World!

**Firefox** Hello, World!

じゃあ、これは？

```
const hello = "Hello";  
const world = "World";  
console.log(`${hello}, ${world}!`);
```

Chrome Hello, World!

Safari Hello, World!

Firefox Hello, World!



## アロー関数でも

```
const log = () => console.log("Hello, World!");  
log();
```

Chrome Hello, World!

Safari Hello, World!

Firefox Hello, World!

全部のブラウザで  
同じ結果になる。  
— これは偶然？

# エンジンはそれぞれ別物

同じ ECMAScript を実装していても、中身は完全に独立したプロダクト。

| Browser       | Engine         | 開発      |
|---------------|----------------|---------|
| Chrome / Edge | V8             | Google  |
| Safari        | JavaScriptCore | Apple   |
| Firefox       | SpiderMonkey   | Mozilla |

別チーム・別コードベース。なのに同じ結果を返す。

偶然じゃなくて、  
すべてのエンジンが参照する  
共通の"物差し"がある。

その物差しが

# Test262

TC39 が保守する ECMAScript 公式コンFORMANCEテストスイート。

規模感

50,000+

個別のテストファイル

As of May 2025 · tc39/test262

主要エンジンはすべてこのテストを CI に組み込み、  
日々の開発で仕様からの退行を検出している。

# 02

Test262 の中身を  
見てみる

# Test262 が対象とする仕様

ECMA-414 Standards Suite にまとめられた 3つの仕様。

**ECMA-262**    言語コア（構文・型・組み込みオブジェクト）

**ECMA-402**    Intl オブジェクトなどの国際化 API

**ECMA-404**    JSON のデータ交換フォーマット

---

DOM や fetch などの Web API は Test262 の対象外。  
こちらは WPT（Web Platform Tests）がカバー。



少しだけ、歴史の話

## 競合していたベンダーが テストを持ち寄って始まった

2010年夏に開発開始、ES5.1（2011年6月）と前後して整備。

# Test262 の源流

## Google

V8 の仕様準拠検証用に開発していた  
Sputnik テストスイート（5,000件超）を提供。

## Microsoft

自社の ES5Conform テストスイートと、  
追加テスト群を提供。

---

熾烈に競合していた2社が自社のテスト資産を持ち寄り、**共通の評価基準**を作った。  
— Web 標準化の歴史における、競争から協調への転換点。

# テストファイルの構造

各テストは **メタデータ** と **テスト本体** の2パート構成。

```
/*---  
esid: sec-array.prototype.includes  
description: Array.prototype.includes should find the target value  
features: [Array.prototype.includes]  
---*/  
  
var arr = [1, 2, 3];  
  
assert.sameValue(arr.includes(2), true);  
assert.sameValue(arr.includes(4), false);
```

— 上部のコメントブロックがメタデータ。ファイル1つにつき、検証する仕様挙動は1つが原則。

# メタデータの主要キー

## **esid**

仕様書のどのセクションを検証しているかの識別子。

例: `sec-array.prototype.includes` — 仕様書のアルゴリズムと 1:1 対応。

## **features**

テストの実行に必要な ECMAScript 機能の宣言。

未実装機能に依存するテストを、エンジン側がスキップできる。

## **flags**

実行モードの制御（`onlyStrict` / `noStrict` など）。

指定なしなら Strict・非 Strict 両方で実行される。

## **description**

人間向けの短い説明。

# アサーション

Test262 は独自の小さなアサーションライブラリを提供。

```
// 値の厳密な等価性
assert.sameValue(actual, expected);

// 特定の例外型のスローを検証（種類まで厳密にチェック）
assert.throws(TypeError, () => {
  null.property; // TypeError でなければ失敗
});
```

「エラーが投げられれば OK」ではない。

仕様が `TypeError` を要求する箇所で `ReferenceError` が出たら、それはテスト失敗。

# こんな粒度でテストされている

同じ「配列内の検索」でも、内部で使う等価判定アルゴリズムが違う。

```
// SameValueZero — Array.prototype.includes  
[NaN].includes(NaN) // → true  
  
// Strict Equality (===) — Array.prototype.indexOf  
[NaN].indexOf(NaN) // → -1
```

仕様アルゴリズム単位でテストファイルが分かれている。

`built-ins/Array/prototype/includes/samevaluezero.js` と  
`built-ins/Array/prototype/indexOf/15.4.4.14-5-13.js` は別物。

— エンジンが片方を実装ミスしたら、ピンポイントでそのテストだけが落ちる。

# 月またぎや繰り上がりまで 仕様で決まっている

エンジンが"勝手に"挙動を決められない代表例が `Date` 。

```
// 2 月 30 日 → 仕様は "3 月 2 日に繰り上げる" と決めている
```

```
new Date(2025, 1, 30).toDateStrin()
```

```
// → "Sun Mar 02 2025"
```

```
// 不正な月 → Invalid Date
```

```
new Date("2025-13-01").getTime() // → NaN
```

```
// NaN を渡せば NaN が返る、ということまで仕様が指定
```

```
new Date(NaN).getTime() // → NaN
```

# Temporal は 使える暦まで仕様で決めている

ECMA-402 の Era and MonthCode Proposal が、14 種類の暦を列挙している。

gregory · japanese · hebrew · chinese · dangi · persian  
islamic-civil · islamic-tbla · islamic-umalqura  
buddhist · coptic · ethioaa · ethiopic · indian · roc  
+ iso8601 (ECMA-262 で必須)

それぞれの暦の era ・ 閏月 ・ 閏年規則までが仕様で決まっていて、  
Test262 はそのすべてに対して個別のテストを持っている。



# Hebrew の閏月まで Test262 が見ている

```
// test/intl402/Temporal/PlainDate/prototype/monthCode/  
// leap-months-hebrew.js  
const commonMonths = ["M01","M02","M03","M04","M05",    "M06","M07","M08","M09","M10","M11","M12"];  
const leapMonths   = ["M01","M02","M03","M04","M05","M05L","M06","M07","M08","M09","M10","M11","M12"];  
  
for (var year = 5730; year < 5735; year++) {  
  const at = (m) => Temporal.PlainDate.from(  
    { year, month: m, calendar: "hebrew", day: 1 });  
  const monthsInYear = at(1).monthsInYear;  
  for (var month = 1; month < monthsInYear; month++) {  
    const expected = at(month).inLeapYear ? leapMonths : commonMonths;  
    assert.sameValue(at(month).monthCode, expected[month - 1]);  
  }  
}
```

— Hebrew 暦の閏年は **M05L** (Adar I) が 5 月と 6 月の間に挿入される。年ごとに 5 年分、月コードの並びを丸ごと検証している。

# 新機能が仕様に入るには テストが必要

TC39 のステージプロセスでは、Stage 2.7 でテスト執筆が本格化し、Stage 4 への進行条件に「2つの互換な実装が Test262 acceptance tests を通過していること」が含まれる。



テストは 提案者・実装者・コミュニティの誰でも書ける。

## Stage 2.7 — テストで設計を検証する

Stage 2.7 は 2024 年に TC39 プロセスに正式導入された比較的新しいステージ。

「設計は原則承認された。あとはテスト・実装・利用からのフィードバックで詰めるのみ」という宣言。

|              |                                                       |
|--------------|-------------------------------------------------------|
| 入る条件         | 完全な仕様テキスト／指定レビュアーと editor group の sign-off が揃った状態。    |
| ここでやること      | 包括的な Test262 テストの作成と、実装可能性を検証する仕様準拠プロトタイプの開発。         |
| Stage 3 との違い | Stage 3 は実装経験を積む段階。2.7 はその前提として、実装者が頼りにできるテストを用意する段階。 |

テストを書く → 実装者がそれを頼りに実装する → 2つの実装が通れば Stage 4

Test262 は  
事後的なテストツールではなく、  
言語仕様の策定プロセス  
そのものに組み込まれている。

# 03

各エンジンは  
どう動かしているか

# 各エンジンが Test262 を実行する方法

すべての主要エンジンが CI に統合。ローカルでも実行できる。

```
$ ./tools/run-tests.py --outdir=out/arm64.release \
  test262/language/expressions/call/eval-spread
```

```
>>> Running tests for arm64.release
```

```
>>> Running with test processors
```

```
[00:25|% 0|+ 2|- 0]: Done
```

```
>>> 56610 base tests produced 2 (0%) non-filtered tests
```

```
>>> 2 tests ran
```

## test262.status — 既知の失敗リスト

実装が仕様に追いついていない箇所は、対応する Test262 テストが落ちる。

→ 失敗が分かっているテストは明示的に登録して CI を通す。

```
# https://bugs.chromium.org/p/v8/issues/detail?id=5690
```

```
'language/expressions/call/eval-spread': [FAIL],
```

test262.status は、エンジンの技術的負債の一覧表。

将来修正されるべき仕様との乖離が、ここに記録されている。

# test262.fyi — 日次で公開される準拠状況

各エンジンの Test262 準拠状況は <https://test262.fyi> で毎日公開されている。  
ナイトリービルドに対してテストを走らせ、機能サポートの差分を可視化するサイト。

- V8 / SpiderMonkey / JavaScriptCore の相互比較
- Ladybird の LibJS のような新興エンジンも掲載
- 準拠度をプロジェクトの進捗指標として活用

可視化 → エンジン間の健全な競争 → Web 全体の相互運用性の底上げ。



# 04

おまけ：  
V8 で見つけたバグの話

# 直接 eval と 間接 eval

eval は呼び出し方で挙動が変わる、という前提の話。

直接 eval — ローカルスコープが見える

```
function f() {  
  const x = 10;  
  eval("console.log(x)");  
  // → 10  
}
```

間接 eval — グローバルスコープで実行

```
function g() {  
  const x = 10;  
  const e = eval;  
  e("console.log(x)");  
  // → ReferenceError  
}
```

ECMAScript 仕様で明確に定義されている挙動。エンジンは正しく判定する義務がある。

## eval(...args) が壊れていた

V8 では、末尾にスプレッドがある eval 呼び出しだけ、直接 eval として認識されていなかった。

```
const args = ["1 + 2"];  
eval(...args);  
// 仕様上は直接 eval のはず  
// → V8 は間接 eval として実行していた
```

# ソースコードには FIXME が残っていた

v8/src/interpreter/bytecode-generator.cc

```
// FIXME(v8:5690): Support final spreads for eval.
```

そして先ほどの `test262.status` には、同じバグ番号でテストが FAIL として登録されていた。

```
# https://bugs.chromium.org/p/v8/issues/detail?id=5690  
'language/expressions/call/eval-spread': [FAIL],
```

— FIXME と test262.status の FAIL。技術的負債の両側からの記録。

# 直したパッチの中身

末尾スプレッドの `eval(...iter)` も `%reflect_apply` ルートに通し、第 1 引数を取り出して直接 `eval` として解決できるようにする。

```
- // FIXME(v8:5690): Support final spreads for eval.
- if (spread_position == Call::kHasNonFinalSpread) {
+ // For direct eval calls with a final spread like eval(...iter), we
+ // also use %reflect_apply so we can extract the first argument
+ // for ResolvePossiblyDirectEval.
+ const bool eval_with_final_spread =
+   spread_position == Call::kHasFinalSpread &&
+   expr->is_possibly_eval() && expr->arguments()->length() > 0;
+ const bool use_reflect_apply =
+   spread_position == Call::kHasNonFinalSpread || eval_with_final_spread;
+ if (use_reflect_apply) {
```

# Test262 が果たした役割

ここで見ていただきたいのは、V8 の内部実装の詳細ではなく、**その裏で Test262 が何をしていたか**の方です。

- 仕様と実装のギャップが、「**テストの失敗**」という形で可視化されている
- `test262.status` に FAIL が並んでいるから、**どの仕様に違反しているか**が一目でわかる
- 修正したあとは、Test262 テストが通ること自体が **仕様準拠の証明**になる

---

発見 → 修正 → Test262 で証明

このサイクルが、JavaScript の「どのブラウザでも同じ結果になる」を支えている。



**Amemiya Riya** ✓

@Riya31377928



🔄 翻訳を表示

Node.js v26 is out 🎉

V8 has been updated to v14.6, which means my V8 fix for `eval(...args)` (direct eval with spread) is now shipping in Node.js 🎉

For years V8 was the only engine mishandling `eval(...iter)` as indirect eval — now it's correctly treated as direct eval



**Rafael Gonzaga** ✓ @\_rafaelgss · 5月5日

Node.js v26.0.0 is out ❤️

Temporal API enabled by default, V8 14.6, Undici 8, and key deprecations as we keep modernizing the platform.

...

# 05

まとめ



# JavaScript の"当たり前"を支える仕組み

- TC39 の合議による仕様策定
- 5万件以上のテストを日々保守するコミュニティ
- 仕様準拠を目指すエンジン開発者たち

---

Test262 は単なるテストツールではない。

競合するベンダーが同一の仕様を正しく実装するための、  
合意形成の基盤

# Test262 は、 誰にでも開かれている。

テスト追加も、エンジンの FIXME 修正も、特別な権限はいりません。  
Proposal にテストを書く。コードリーディングでギャップを見つける。  
それは Web の未来の"当たり前"を作る直接的な貢献です。

# ご清聴ありがとうございました



スライド: [github.com/riya-amemiya/amemiya\\_riya\\_slide\\_data](https://github.com/riya-amemiya/amemiya_riya_slide_data)

SNS等: [riya-amemiya-links.tokidux.com](https://riya-amemiya-links.tokidux.com)

---

このスライドは CC BY-SA 4.0 でライセンスされています。  
より自由な翻訳を可能にするため、翻訳は例外的に CC BY 4.0 での配布が許可されています。  
Required Attribution: Riya Amemiya (<https://github.com/riya-amemiya>)